

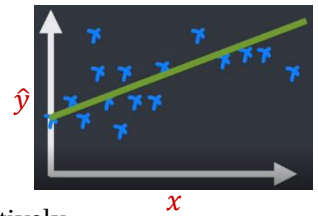
Regression – Linear and Polynomial Regression

Chapter 4: Training Models

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Linear Regression

- In statistics, linear regression mathematically models the relationship between independent and dependent variables.
- In machine learning and data science, linear regression represents the relationship between the feature (independent variable) and the target (dependent variable) as a linear equation, which can then be used to make predictions.
- Linear regression is one of the simplest supervised learning algorithms used for regression tasks where the target is a quantitative value.
- Types: [Univariate Linear Regression](#), and [Multivariable Linear Regression](#).

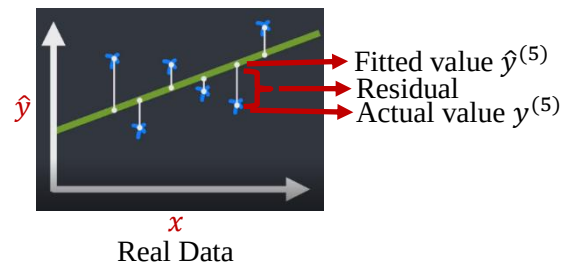
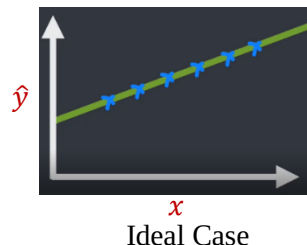
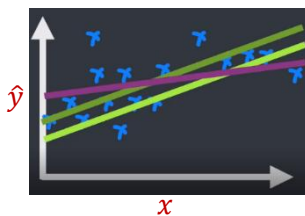


Univariate Linear Regression

- Univariate linear regression works for a single feature/input.
- A univariate linear function with input (feature) x and output (target) y has the form $\hat{y} = \theta_0 + \theta_1 x$, where θ_0 and θ_1 are real-valued coefficients to be learned.
- It is like fitting a straight line where y-intercept and slope are analogous to θ_0 and θ_1 respectively.
- θ_0 and θ_1 are often referred to as w_0 and w_1 , we think of the coefficients as weights; the value of $\hat{y}$ is changed by changing the relative weight of one term or another.
- $\hat{y}$ is often referred to as $h(x)$, where h stands for hypothesis, hence, $\hat{y} = h(x) = \theta_0 + \theta_1 x$.

Choosing the Best Fitting Line (θ_0 and θ_1)

- The goal in linear regression is to find the line that best fits the data by choosing optimal values for θ_0 and θ_1 .
 - **Ideal Case:** A single line perfectly passes through all training points (no error).
 - **Real Data:** A perfect fit is rare. The difference between predicted and actual values, called **residuals**, determines how good the fit is.



- To evaluate the fit, we minimize the error between predictions $\hat{y}$ and actual outputs y .
- The total error is measured using squared-error loss function, summed over all the training examples:
 $\sum_{i=1}^n (y^{(i)} - h(x^{(i)}))^2$ or $\sum_{i=1}^n (y^{(i)} - \theta_0 - \theta_1 x^{(i)})^2$, where n is the number of data points.
 - Called **Sum of Squared Errors (SSE)** or **Residual Sum of Squares (RSS)**.
- Linear regression aims to minimize RSS, which forms the **cost function**.

Several Possible Cost Functions

1. Mean Squared Error (MSE)

- $MSE = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$
- Standard choice. Smooth, differentiable, used for gradient descent.

2. Mean Absolute Error (MAE)

- $MAE = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|$
- More robust to outliers, easier to interpret but less smooth for optimization.

3. Root Mean Square Error (RMSE)

- $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}$

Gradient Descent for Univariate Linear Regression

- Feature: x (1 feature)
- Target: y
- Hypothesis: $h(\theta) = \hat{y} = \theta_0 + \theta_1 x$
- Learning parameters: θ_1 and θ_0
- Cost function: $J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \theta_0 - \theta_1 x^{(i)})^2$
- Objective: $\min(J(\theta_0, \theta_1))$ i.e. $y \approx \hat{y}$
- Partial derivative of the cost function w.r.t θ_0 and θ_1 :

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0} = -\frac{2}{n} \sum_{i=1}^n (y^{(i)} - \theta_0 - \theta_1 x^{(i)}) = \frac{2}{n} \sum_{i=1}^n (\theta_0 + \theta_1 x^{(i)} - y^{(i)}) = \frac{2}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})$$

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} = -\frac{2}{n} \sum_{i=1}^n (y^{(i)} - \theta_0 - \theta_1 x^{(i)}) \cdot x^{(i)} = \frac{2}{n} \sum_{i=1}^n (\theta_0 + \theta_1 x^{(i)} - y^{(i)}) \cdot x^{(i)} = \frac{2}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \cdot x^{(i)}$$
- The algorithm:
Repeat until converge:

$$t_0 = \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \theta_0 - \alpha \left(\frac{2}{n} \sum_{i=1}^n (\theta_0 + \theta_1 x^{(i)} - y^{(i)}) \right) = \theta_0 - \alpha \left(\frac{2}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \right)$$

$$t_1 = \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \theta_1 - \alpha \left(\frac{2}{n} \sum_{i=1}^n (\theta_0 + \theta_1 x^{(i)} - y^{(i)}) \cdot x^{(i)} \right) = \theta_1 - \alpha \left(\frac{2}{n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \cdot x^{(i)} \right)$$

$$\theta_0 = t_0$$

$$\theta_1 = t_1$$
- θ_0 is called the bias. A bias shifts the curve along any axis, in case of linear regression, it is always the y-axis.
- θ_1 represents the weight of the associated feature in the final decision making. It could be negative too.

Evaluating Regression model

The R-Squared Metric – Goodness of Fit for Regression

- R-squared, also called the **coefficient of determination**, measures how well a linear regression model explains the variation in the dependent variable.

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^n (y^{(i)} - \bar{y})^2}$$
- RSS (*residual sum of squares*) = $\sum_{i=1}^n (y^{(i)} - h(x^{(i)}))^2$, measures unexplained variance (error).
- TSS (*total sum of squares*) = $\sum_{i=1}^n (y^{(i)} - \bar{y})^2$, where $\bar{y}$ is the mean value of the training data, measures total variance in the data.
- R^2 tells us what percentage of the variance in the output is explained by the model.
- Range: Can go from $-\infty$ to 1.
 - $R^2 = 1$: Perfect fit.
 - $R^2 = 0$: Model is no better than predicting the mean $\bar{y}$ (a flat horizontal line).
 - $R^2 < 0$: Model is worse than the mean predictor (can happen if the model is poorly fitted or lacks a constant term).
- Higher R^2 : Better fit, as more variance in data is captured by the model.

Polynomial Regression

- Applying linear model on a linear dataset gives good results using simple linear regression.
- But applying same model without any modification on a non-linear dataset will increase error rate and thus reduce accuracy.
- So for such cases, where data points are arranged in a non-linear fashion, Polynomial Regression model is used.
- A simple way to do this is to add powers of each feature as new features, then train a linear model on this extended set of features.
- Polynomial Regression is a regression algorithm that models the relationship between a dependent and independent variable as n^{th} degree polynomial.
- The polynomial regression equation is given below:
$$h(x) = \hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^3 + \dots + \theta_m x_1^m, \quad \text{where } m \text{ is the number of features.}$$

Adding Non-linear Features

- Form of hypothesis function can be improved using polynomial regression, if the hypothesis function does not fit the data well.
- The behavior or curve of the hypothesis function can be changed by making it a quadratic, cubic or square root function (or any other form).
- The polynomial hypothesis functions usually provide a better fit on non-linear data, but are computationally expensive as they increase the number of parameters.
- This diagram also illustrates the fact that training a model means searching for a combination of model parameters that minimizes a cost function (over the training set). It is a search in the model's parameter space: the more parameters a model has, the more dimensions this space has, and the harder the search is: searching for a needle in a 300-dimensional haystack is much trickier than in three dimensions. Fortunately, since the cost function is convex in the case of Linear Regression, the needle is simply at the bottom of the bowl.